ANALITYKA GOSPODARCZA / DATA MINING

Laboratorium 1

1. Import bibliotek i obiekt Path

Na początku importujemy narzędzia do pracy z plikami, tabelami i wykresami.

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt

sciezka = Path("zamowienia_sklep.csv")
print(sciezka.exists())
```

- Path reprezentuje ścieżkę do pliku; exists() zwraca True, jeśli plik istnieje.
- pandas to biblioteka do pracy z tabelami; pd to jej zwyczajowy skrót.
- matplotlib.pyplot pozwala tworzyć wykresy; plt to zwyczajowy skrót.

Kiedy używać: Przed wczytaniem i badaniem zbioru danych.

Na co uważać: Ścieżka względna jest liczona od bieżącego katalogu pracy kernela. Gdy pliku nie znaleziono, najpierw sprawdź katalog i nazwę pliku.

2. Wczytanie CSV i opcje wyświetlania

CSV przechowuje dane tabelaryczne w wierszach i kolumnach. Funkcja pd.read_csv() tworzy DataFrame.

```
zamowienia = pd.read_csv("zamowienia_sklep.csv")
pd.set_option("display.max_columns", 25)
zamowienia.head()
```

- DataFrame to obiekt tabelaryczny; każda kolumna ma nazwę i typ.
- set_option() zmienia sposób pokazywania wyników, a nie same dane.
- head() zwraca pierwszych pięć wierszy; head(8) zwróci osiem.

Kiedy używać: Na początku audytu, aby potwierdzić, czy plik odczytano poprawnie.

Na co uważać: Dla polskich CSV separator może być średnikiem: pd.read_csv("plik.csv", sep=";"). Sprawdź również kodowanie oraz zapis liczb dziesiętnych.

3. Rozmiar, nazwy kolumn i jednostka obserwacji

Najpierw ustal, czym jest jeden wiersz. W analizie zamówień może to być zamówienie, pozycja zamówienia albo zdarzenie w historii.

```
print(zamowienia.shape)
print(zamowienia.columns.tolist())
print(len(zamowienia))
```

- shape zwraca krotkę (liczba_wierszy, liczba_kolumn).
- columns zawiera nazwy kolumn; tolist() zamienia je na zwykłą listę.
- len(zamowienia) podaje liczbę wierszy.

Kiedy używać: Zanim obliczysz udziały, średnie albo liczbę unikalnych obiektów.

Na co uważać: Liczba wierszy nie musi być liczbą unikalnych zamówień. Ustal identyfikator i znaczenie pojedynczego rekordu.

4. Typy danych i przegląd df.info()

info() pokazuje nazwy pól, liczbę niepustych wartości i typy kolumn.

```
zamowienia.info()
print(zamowienia.dtypes)
```

- Typ object często sygnalizuje tekst, lecz może też oznaczać wartości mieszane.
- int64 i float64 opisują liczby, a datetime64 reprezentuje daty po przekształceniu.
- Non-Null Count pozwala szybko zauważyć potencjalne braki.

Kiedy używać: Przed agregacjami, interpretacją dat oraz obliczeniami.

Na co uważać: Kolumna liczb wczytana jako tekst nie będzie zachowywać się jak prawdziwa liczba. Sam wygląd wartości nie wystarcza do oceny typu.

5. Rozkład zmiennej celu: value_counts()

Przy zadaniu przewidywania zwrotu produktu zmienną celu może być kolumna zwrot o wartościach tak i nie.

```
licznosci = zamowienia["zwrot"].value_counts(dropna=False)
procenty = (zamowienia["zwrot"]
            .value_counts(normalize=True, dropna=False)
            .mul(100).round(1))
print(licznosci)
print(procenty)
```

- value_counts() liczy wystąpienia kategorii.
- dropna=False nie ukrywa braków; normalize=True podaje udziały zamiast liczebności.
- .mul(100) zamienia udziały na procenty, .round(1) zaokrągla wynik.

Kiedy używać: Do sprawdzenia, czy badane zdarzenie jest rzadkie i jak wygląda równowaga klas.

Na co uważać: Przy silnym niezrównoważeniu klas sama accuracy może być myląca: przewidywanie najczęstszej klasy dla wszystkich nie rozwiązuje problemu biznesowego.

6. Braki danych: isna(), sum(), sort_values()

Brak danych może oznaczać brak wpisu, niepowodzenie rejestracji lub informację niedotyczącą rekordu.

```
braki = zamowienia.isna().sum().sort_values(ascending=False)
print(braki[braki > 0])
print(braki[braki > 0].to_frame("liczba_brakow"))
```

- isna() tworzy tabelę wartości True/False dla każdej komórki.
- sum() zlicza True jako 1 w każdej kolumnie.
- sort_values(ascending=False) ustawia największe liczniki na początku.
- to_frame() zamienia Series w jedną kolumnę DataFrame o wskazanej nazwie.

Kiedy używać: Do wykrycia, które cechy wymagają osobnej decyzji o czyszczeniu.

Na co uważać: Braki nie są zawsze losowe. Sprawdź, czy częściej występują np. w jednym kanale sprzedaży.

7. Wiersze powtórzone i identyfikatory

Duplikat pełnego rekordu i powtórzenie identyfikatora to różne problemy.

```
print(zamowienia.duplicated().sum())
print(zamowienia["zamowienie_id"].duplicated().sum())
powtorzenia = zamowienia[zamowienia["zamowienie_id"].duplicated(keep=False)]
print(powtorzenia.sort_values("zamowienie_id"))
```

- duplicated() domyślnie oznacza drugi i kolejne identyczne wiersze.
- duplicated(keep=False) oznacza wszystkie wiersze należące do powtarzającej się grupy.
- sort_values() ułatwia wzrokowe porównanie rekordów o tym samym identyfikatorze.

Kiedy używać: Do oceny, czy analizowany plik ma niepożądane powtórzenia.

Na co uważać: Powtarzający się identyfikator nie zawsze jest błędem. Jeśli jeden wiersz opisuje pozycję zamówienia, powtórzenia mogą być oczekiwane.

8. Kategorie: unique(), dropna(), astype(), sorted()

Kategorie potrafią różnić się wielkością liter, spacjami lub literówkami. Przed grupowaniem należy je obejrzeć.

```
for kolumna in ["region_sprzedazy", "kanal", "rodzaj_dostawy"]:
    wartosci = zamowienia[kolumna].dropna().astype(str).unique()
    print(kolumna, sorted(wartosci))
```

- dropna() odrzuca wartości brakujące z listy kategorii.
- astype(str) umożliwia traktowanie pozostałych wartości jako tekst.
- unique() zwraca niepowtarzające się wartości.
- sorted() porządkuje je, co ułatwia zauważenie różnic.

Kiedy używać: Przed wyliczaniem częstości kategorii lub porównaniami regionów.

Na co uważać: Napisy "Online", "online" i "online " to w danych trzy różne wartości. Nie poprawiaj wszystkich kategorii automatycznie bez wiedzy o ich znaczeniu.

9. Statystyki opisowe: describe()

describe() pomaga szybko ustalić zakres i podstawowy rozkład cech liczbowych.

```
kolumny = ["wartosc_zamowienia", "dni_dostawy", "ocena_obslugi"]
print(zamowienia[kolumny].describe())
```

- count to liczba dostępnych obserwacji, mean średnia, std odchylenie standardowe.
- min i max pokazują wartości skrajne; 25%, 50%, 75% to kwartyle.
- Mediana to 50%: połowa niepustych obserwacji nie przekracza jej wartości.

Kiedy używać: Do wychwycenia podejrzanych zakresów i porównania skali zmiennych.

Na co uważać: Statystyki nie powiedzą, czy obserwacja jest błędem. Nietypowa wartość może być prawdziwym, ważnym przypadkiem.

10. Wyszukiwanie rekordów naruszających reguły

Filtr logiczny wybiera te wiersze, które spełniają warunek. Operatory & i | łączą warunki kolumnowe.

```
bledne = zamowienia[
    (zamowienia["dni_dostawy"] < 0)
    | (zamowienia["ocena_obslugi"] > 5)
]
print(bledne[["zamowienie_id", "dni_dostawy", "ocena_obslugi"]])
```

- Każde porównanie tworzy maskę True/False dla wszystkich wierszy.
- | oznacza LUB, & oznacza ORAZ w filtrowaniu pandas.
- Zewnętrzne nawiasy kwadratowe wybierają wiersze, dla których maska ma wartość True.

Kiedy używać: Do znajdowania rekordów, które łamią reguły biznesowe.

Na co uważać: Otaczaj każde porównanie nawiasami. Wartość brakująca wymaga osobnej uwagi, ponieważ porównania nie zawsze ją wychwycą.

11. Łączenie wykrytych problemów: concat() i assign()

Można przygotować wspólną tabelę problematycznych rekordów i dodać opis przyczyny.

```
za_dlugo = zamowienia[zamowienia["dni_dostawy"] > 60].assign(
    problem="nietypowy czas dostawy")
ujemne = zamowienia[zamowienia["wartosc_zamowienia"] < 0].assign(
    problem="ujemna wartość")
raport = pd.concat([za_dlugo, ujemne], ignore_index=True)
print(raport[["zamowienie_id", "problem"]])
```

- assign() tworzy lub aktualizuje wskazaną kolumnę w wyniku operacji.
- pd.concat() dokleja tabele wzdłuż wierszy, jeśli axis nie zostało zmienione.
- ignore_index=True nadaje nową numerację indeksu wynikowego.

Kiedy używać: Do sporządzania listy rekordów wymagających wyjaśnienia.

Na co uważać: Ten sam wiersz może spełniać kilka reguł i pojawić się w raporcie więcej niż raz. Trzeba zdecydować, czy to pożądane.

12. Tabele kontyngencji: pd.crosstab()

Tabela krzyżowa pokazuje częstości dwóch kategorii i pozwala porównać udziały w grupach.

```
tabela = pd.crosstab(
    zamowienia["rodzaj_dostawy"],
    zamowienia["zwrot"],
    normalize="index"
).round(3)
print(tabela)
```

- Wiersze odpowiadają kategoriom pierwszej zmiennej, kolumny kategoriom drugiej.
- normalize="index" sprawia, że udziały w każdym wierszu sumują się do 1.
- round(3) wyświetla trzy miejsca po przecinku.

Kiedy używać: Do porównania częstości zwrotów między sposobami dostawy.

Na co uważać: Udział 0.20 oznacza 20%, nie 0.20%. Przed interpretacją udziałów sprawdź również liczebności poszczególnych grup.

13. Średnie w grupach: groupby()

groupby() dzieli wiersze według kategorii, a następnie stosuje obliczenie w każdej grupie.

```
srednie = (zamowienia.groupby("zwrot")
           [["wartosc_zamowienia", "dni_dostawy", "ocena_obslugi"]]
           .mean().round(2))
print(srednie)
```

- groupby("zwrot") tworzy grupy, np. tak i nie.
- Podwójne nawiasy kwadratowe [[...]] wybierają kilka kolumn.
- mean() wylicza średnie liczbowych cech osobno w każdej grupie.

Kiedy używać: Do poszukiwania różnic opisowych pomiędzy obserwacjami.

Na co uważać: Średnia może ukrywać skośny rozkład lub odstające wartości. Różnica średnich nie dowodzi przyczynowości.

14. Szybki wykres słupkowy z pandas i matplotlib

Wynik grupowania można pokazać na wykresie, który ułatwia porównania.

```
udzialy = pd.crosstab(zamowienia["kanal"], zamowienia["zwrot"],
                      normalize="index")
udzialy.plot(kind="bar")
plt.ylabel("Udział zamówień")
plt.title("Zwroty w kanałach sprzedaży")
plt.tight_layout()
plt.show()
```

- plot(kind="bar") buduje wykres słupkowy na podstawie DataFrame.
- ylabel() i title() opisują osie i sens wykresu.
- tight_layout() dopasowuje odstępy, a show() wyświetla wykres.

Kiedy używać: Podczas prezentowania różnic między grupami.

Na co uważać: Dwa słupki o podobnej wysokości mogą reprezentować bardzo różne liczebności. Nie pomijaj tego kontekstu.

15. Kiedy kolumna zdradza przyszłość: data leakage

W modelu predykcyjnym wolno użyć tylko informacji znanych w chwili podejmowania decyzji. Kolumna wpisana dopiero po zwrocie produktu nie może wyjaśniać wcześniejszego ryzyka.

```
czy_uzupelniona = zamowienia["data_wyplaty_zwrotu"].fillna("").ne("")
kontrola = pd.crosstab(zamowienia["zwrot"], czy_uzupelniona)
print(kontrola)
```

- fillna("") zastępuje puste pola pustym napisem w tej analizie.
- ne("") oznacza porównanie „różne od pustego napisu”.
- crosstab pozwala sprawdzić, czy dostępność takiej informacji silnie zależy od zdarzenia.

Kiedy używać: Przy audycie kandydatów na cechy przyszłego modelu.

Na co uważać: Wysoka jakość modelu może być pozorna, jeżeli wykorzystuje informacje z przyszłości. Ustal moment prognozy i datę dostępności każdej cechy.

16. Brak równowagi klas a interpretacja jakości

Gdy pozytywne zdarzenie jest rzadkie, model zawsze wybierający najczęstszą odpowiedź może osiągać wysoki procent trafień.

```
licznosci = zamowienia["zwrot"].value_counts()
udzial_najczestszej = licznosci.max() / licznosci.sum()
print(f"Bazowa trafność: {udzial_najczestszej:.1%}")
```

- Taki wynik jest użytecznym punktem odniesienia, a nie dowodem jakości modelu.
- W dalszych etapach rozważa się m.in. precision, recall, F1 oraz koszty decyzji.

Kiedy używać: Przy interpretowaniu zmiennej celu i projektowaniu oceny przyszłej klasyfikacji.

Na co uważać: Nie prezentuj wysokiego accuracy bez informacji o częstości poszczególnych klas i celu biznesowym.

17. Jak formułować wnioski z audytu

W laboratorium rozpoznajemy wzorce, ale nie przeprowadzamy jeszcze eksperymentu ani pełnego modelowania.

```
srednie = zamowienia.groupby("zwrot")["dni_dostawy"].mean()
print(srednie)
```

- Opis: „W analizowanym zbiorze średni czas dostawy był inny w dwóch grupach”.
- Hipoteza: „Dłuższy czas dostawy może wiązać się z większym ryzykiem zwrotu”.
- Nieuprawniony wniosek: „Dłuższa dostawa powoduje zwrot”, jeżeli nie wykazano związku przyczynowego.

Kiedy używać: Gdy przedstawiasz wyniki menedżerom lub przygotowujesz kolejny etap analizy.

Na co uważać: Sprawdź liczebności, jakość danych, możliwe zmienne zakłócające i to, czy różnica jest stabilna.

18. CRISP-DM: od pytania do wykorzystania wyniku

CRISP-DM porządkuje projekt analizy danych. Audyt z pierwszego laboratorium przede wszystkim wspiera rozumienie danych, ale wyniki mają konsekwencje dla całego procesu.

Etap	Pytanie i rezultat
Business Understanding	Jaką decyzję chcemy wesprzeć? Definicja zwrotu i kosztu interwencji.
Data Understanding	Jak wygląda źródło? Raport braków, typów, duplikatów, zakresów i kategorii.
Data Preparation	Jak ujednolicić wartości, obsłużyć braki i wykluczyć dane z przyszłości?
Modeling	Jakie podejścia porównać i jaki wynik bazowy przyjąć?
Evaluation	Czy model rozwiązuje problem i jakie błędy są najbardziej kosztowne?

Deployment	W jaki sposób wynik trafi do procesu, kto go monitoruje i jak będzie aktualizowany?

19. Dane wrażliwe i etyka analizy

- Stosuj zasadę minimalizacji: zbieraj i wykorzystuj tylko informacje uzasadnione celem analizy.
- Sprawdź, czy cechy takie jak wiek, lokalizacja lub dochód nie prowadzą do niesprawiedliwych różnic w traktowaniu grup.
- Zadbaj o legalną podstawę przetwarzania, ograniczenie dostępu i odpowiedni czas przechowywania.
- Przy decyzjach dotyczących osób zaplanuj kontrolę człowieka, możliwość wyjaśnienia wyniku i monitorowanie błędów.

20. Procedura pierwszego audytu: lista kontrolna

Kolejność	Co sprawdzić	Przykładowe narzędzie
1	Czy plik i nazwy kolumn są prawidłowe?	read_csv(), head(), columns
2	Co oznacza wiersz i ile jest rekordów?	shape, len(), nunique()
3	Które dane są liczbami, tekstem lub datą?	info(), dtypes
4	Jak wygląda zmienna celu?	value_counts()
5	Gdzie są braki, duplikaty i niespójne kategorie?	isna(), duplicated(), unique()
6	Które wartości naruszają reguły?	describe(), maski logiczne
7	Czy widać różnice między grupami?	crosstab(), groupby(), plot()
8	Czy cechy były dostępne w chwili prognozy?	audyt data leakage

21. Zadania do samodzielnego wykonania

- Wyświetl liczebności kategorii kanału sprzedaży, uwzględniając braki.
- Wypisz wiersze ze zbyt niską oceną obsługi albo ujemną wartością zamówienia.
- Porównaj średnią wartość zamówienia w grupie ze zwrotem i bez zwrotu.
- Sprawdź, czy dowolna kolumna z informacją o zakończonej reklamacji byłaby dostępna w chwili prognozy.

22. Jak czytać zapis pandas od lewej do prawej

W kodzie analitycznym najwięcej trudności sprawia często nie pojedyncza metoda, lecz kilka operacji połączonych w jednej instrukcji. Dobrym sposobem jest przypisanie każdemu fragmentowi roli: wybór kolumny, utworzenie maski, wybranie wierszy i wykonanie agregacji.

```
warunek = zamowienia["wartosc_zamowienia"] >= 100
wieksze = zamowienia.loc[warunek, ["zamowienie_id", "kanal"]]
liczby = wieksze["kanal"].value_counts(dropna=False)
print(liczby)
```

- zamowienia["wartosc_zamowienia"] wybiera pojedynczą kolumnę jako Series.
- Porównanie >= 100 tworzy serię wartości True/False o tej samej długości co dane.
- loc[warunek, ["zamowienie_id", "kanal"]] ogranicza jednocześnie wiersze i kolumny.
- value_counts(dropna=False) zlicza kanały tylko w wybranym podzbiorze i zachowuje informacje o brakach.

Warto sprawdzać pośrednio warunek.head(), wieksze.shape oraz wieksze.head(). Jeżeli końcowy wynik jest zaskakujący, łatwiej znaleźć etap, na którym powstał problem.

23. Przejście przez kompletny mini-audyt

Poniższa tabela jest tworzona bezpośrednio w Pythonie, więc można ją uruchomić nawet bez pliku CSV. Celowo umieszczono w niej duplikat pełnego rekordu, brak kategorii, niepoprawną ocenę oraz dwie wersje nazwy kanału sprzedaży.

```
import pandas as pd
zakupy = pd.DataFrame({
    "id": [501, 502, 503, 503, 504, 505],
    "kanal": ["Online", "online ", "Salon", "Salon", None, "Online"],
    "ocena": [5, 4, 7, 7, None, 2],
    "zwrot": ["nie", "nie", "tak", "tak", "nie", "tak"],
    "kwota": [90, 160, 210, 210, 45, 70]
})
print("Wymiary:", zakupy.shape)
print("Pełne kopie:", zakupy.duplicated().sum())
print("Braki:", zakupy.isna().sum())
print("Kanały:", zakupy["kanal"].dropna().unique())
print("Oceny poza skalą:", (~zakupy["ocena"].between(1, 5) & zakupy["ocena"].notna()).sum())
```

Wyniki, które należy umieć uzasadnić: 6 wierszy i 5 kolumn; 1 pełny duplikat; 1 brak kanału i 1 brak oceny; kategorie „Online”, „online ” i „Salon”; dwie nieprawidłowe oceny 7, ponieważ wadliwy wiersz występuje dwukrotnie.

Co zmieni się po usunięciu pełnego duplikatu? Pozostanie 5 wierszy. Liczba błędnych ocen spadnie do 1, ale bez dodatkowego działania ocena 7 nadal będzie obecna. Usunięcie duplikatów nie jest tym samym co poprawianie wartości.

24. Kiedy podobne liczby prowadzą do innych wniosków

Sytuacja	Co oznacza wynik	Co sprawdzić dalej
10 braków w kolumnie	Dziesięć rekordów nie zawiera wartości.	Czy brak zależy od grupy, czasu lub źródła?
10 powtórzeń ID	Ten sam identyfikator wystąpił ponownie.	Czy wiersz to transakcja, pozycja, czy zdarzenie?
10% zwrotów	Jedna dziesiąta obserwacji należy do klasy tak.	Liczebności, populację i okres obserwacji.
Wyższa średnia w grupie	Grupy różnią się opisowo.	Rozkład, liczebność, zmienne zakłócające.
Silny związek z datą zdarzenia	Data może niemal zdradzać etykietę.	Czy data była dostępna w chwili predykcji?